Information Retrieval

WS 2022 / 2023

Lecture 9, Tuesday January 10th, 2023

(Latent Semantic Indexing)

Prof. Dr. Hannah Bast
Chair of Algorithms and Data Structures
Department of Computer Science
University of Freiburg

UNI FREIBURG

Overview of this lecture

- Organizational
  - Your experiences with ES8 — Vector Space Model (VSM)
  - Date of the written exam — Feb 28, register until Jan 15
- Contents
  - Latent Semantic Indexing (LSI) — Motivation + examples
  - Singular Value Decomp. (SVD) — Definition + how to compute
  - Using LSI for retrieval — Three variants
  - ES9: a "pencil and paper" sheet, for understanding and practicing the linear algebra (in particular, the SVD)

    In past years, we used to have an implementation exercise for LSI. This was nice, because one could see the results. But it turned out that most people coded it without understanding how it really works.

Experiences with ES9 (VSM)

- Summary / excerpts
  - Gentle introduction to linear algebra, relatively easy (for most, but not all), efficient normalization was a bit tricky

    "I enjoyed this sheet a lot. It was a very good way of getting back into linear algebra."

    "My linear algebra is a little bit rusty but I hope I can brush it up in the upcoming lectures"

    "I spend most of the time debugging [sparse matrix stuff]"

    "Most of the time used for efficient normalization"

    "I found it difficult to find the right numpy functions"

    "If I had a friendship with scipy and numpy, it is over now"

    Remember: there is a numpy/scipy "cheat sheet" on the Wiki!

Exam

- There will be a written exam
  - **February 28** (Tuesday), **11:00 – 13:xx h**

    The exact duration is not yet fixed, so please plan for 2.5 hours though it might be less
  - Rooms: **SR 101-0-10/14** and **SR 101-1-9/13**

    This is the room of the lecture and the room in the floor below, both in building 101

    It will be announced in time, in which room **you** will sit

    More information about the exam is provided in the last lecture (Lecture 13), on February 7

    Registration deadline for the exam: January 15

Latent Semantic Indexing 1/9

- Motivation
  - Let's look at our example toy collection from L8 again:

    D_1 and D_2 and D_3 are "about" surfing the web

    D_5 and D_6 are "about" surfing on the beach

    internet and web are **synonyms**, surfing is a **polysem**
    = means different things in different context

	D_1	D_2	D_3	D_4	D_5	D_6
internet	1	1	0	1	0	0
web	1	0	1	1	0	0
surfing	1	1	1	2	1	1
beach	0	0	0	1	1	1

Latent Semantic Indexing 2/9

- Motivation
  - Let's look at the query web surfing again, using dot-product similarity as explained in L8
  - Then $sim(D_3, Q) > sim(D_2, Q) = sim(D_5, Q)$

    But D_2 seems just as relevant for the query as D_3, only that the word "internet" is used instead of "web"

	REL	REL	REL	(REL)	NOT	NOT	
	D_1	D_2	D_3	D_4	D_5	D_6	Q
internet	1	1	0	1	0	0	0
web	1	0	1	1	0	0	1
surfing	1	1	1	2	1	1	1
beach	0	0	0	1	1	1	0
	2	1	2	3	1	1	

too low

Latent Semantic Indexing 3/9

- Conceptual solution

	REL	REL	REL	(REL)	NOT	NOT	
	D_1	D_2	D_3	D_4	D_5	D_6	Q
internet	1	1	**1**	1	0	0	0
web	1	**1**	1	1	0	0	1
surfing	1	1	1	2	1	1	1
beach	0	0	0	1	1	1	0
	2	2	2	3	1	1	

Add the missing synonyms to the documents

Then indeed: $sim(D_1, Q) = sim(D_2, Q) = sim(D_3, Q)$

The goal of LSI is to do something like this **automagically**

Latent Semantic Indexing 4/9

- A simple but powerful observation

	D_1 (B_1)	D_2 (B_1)	D_3 (B_1)	D_4 (B_1+B_2)	D_5 (B_2)	D_6 (B_2)
internet	1	1	**1**	1	0	0
web	1	**1**	1	1	0	0
surfing	1	1	1	2	1	1
beach	0	0	0	1	1	1

B_1	B_2
1	0
1	0
1	1
0	1

The modified matrix has **column rank 2**

That is, we can write each column as a (different) linear combination of the same two base columns (B_1 and B_2)

Note 1: the original matrix had column rank 4

Note 2: one can prove that **column rank = row rank**

Latent Semantic Indexing 5/9

- Matrix factorization

4 x 2

B_1	B_2
1	0
1	0
1	1
0	1

•

2 x 6

D'_1	D'_2	D'_3	D'_4	D'_5	D'_6
1	1	1	1	0	0
0	0	0	1	1	1

=

$B_1 + B_2$

4 x 6

D_1	D_2	D_3	D_4	D_5	D_6
1	1	**1**	1	0	0
1	**1**	1	1	0	0
1	1	1	2	1	1
0	0	0	1	1	1

Equivalently: the 4 x 6 term-document matrix can be written as a product of a 4 x 2 matrix with a 2 x 6 matrix

The base vectors B_1 and B_2 are the underlying **"concepts"**

The vectors $D'_1, \ldots, D'_6$ are the representation of the documents in the (lower-dimensional) **"concept space"**

Latent Semantic Indexing 6/9

- The goal of LSI
  - Given an $m \times n$ term-document matrix A and $k < \text{rank}(A)$
  - Then find a matrix A' of (column) rank k such that the difference between A' and A is **as small as possible**
  - Formally: $A' = \text{argmin}_{A' \text{ m x n with rank k}} \| A - A' \|$
  - For the $\| \ldots \|$ we take the so-called **Frobenius norm**

    This is defined as $\| D \| := \text{sqrt}(\Sigma_{ij} D_{ij}^2)$

    The reason for using exactly this norm is purely technical: that way, the math on the next slides works out nicely

Latent Semantic Indexing 7/9

- How to compute such an approximation A'
  - We first compute the so-called **singular value decomposition (SVD)** of the given matrix A :
  - **Theorem:** for any m x n matrix A of rank r, there exist U, S, V such that $\mathbf{A} = \mathbf{U} \cdot \mathbf{S} \cdot \mathbf{V}$, and where

    $r \leq \min(m, n)$

    $m \times n \quad m \times r \quad r \times r \quad r \times n$

    U is an **m x r** matrix with $U^T \cdot U = I_r$ $\quad I_r$ = the r x r identity matrix

    S is an **r x r** matrix with nonzero entries only on its diagonal

    V is an **r x n** matrix with $V \cdot V^T = I_r$
  - The decomposition is **unique** up to simultaneous permutation of the rows/columns of U, S, and V

    Standard form: diagonal entries of S positive and sorted largest first

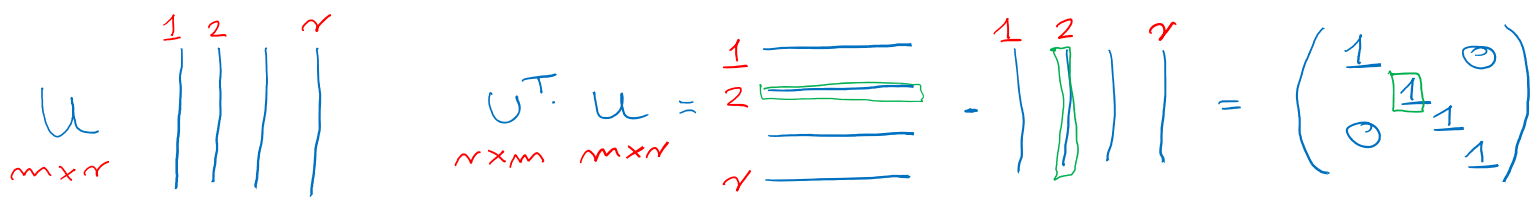
Latent Semantic Indexing 8/9

$$(x_1, \dots, x_n) \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix} = \sum_{i=1}^{m} x_i^2$$

- Understanding $U^T \cdot U = I_r$ and $V \cdot V^T = I_r$
  - Understand that $U^T \cdot U = I_r$ is equivalent to this statement:

    The **columns** of **U** are L_2-normalized and pairwise orthogonal
  - Analogously, $V \cdot V^T = I_r$ is equivalent to this statement:

    The **rows** of **V** are L_2-normalized and pairwise orthogonal
  - Understand that matrix multiplication is not commutative and $\underset{m \times r}{U} \cdot \underset{r \times m}{U^T}$ and $\underset{m \times r}{V^T} \cdot \underset{r \times m}{V}$ are something completely different



Latent Semantic Indexing 9/9

- Using the SVD, our task becomes easy
  - Let $A = U \cdot S \cdot V$ be the **SVD** of A
  - For a given $k < \text{rank}(A)$ let

    $\mathbf{U_k}$ = the first k columns of U — now an m x k matrix

    $\mathbf{S_k}$ = the upper k x k part of S — now a k x k matrix

    $\mathbf{V_k}$ = the first k rows of V — now a k x n matrix
  - Note that U_k is "column-orthonormal" just like U

    That is, $U_k^T \cdot U_k = I_k$ … and similarly, $V_k \cdot V_k^T = I_k$
  - Let $\underset{m \times n}{\mathbf{A_k}} = \underset{m \times k}{\mathbf{U_k}} \cdot \underset{k \times k}{\mathbf{S_k}} \cdot \underset{k \times n}{\mathbf{V_k}}$

    Then A_k is a matrix of rank k that minimizes $\|A - A_k\|$

    This requires a proof, which we don't do in this lecture

Computing the SVD 1/7

- Overview
  - Mathematically, the SVD be computed from the **eigenvalue decomposition** (often just called: eigendecomposition)

    See the next slides for some of the mathematics behind

  - In practice, numerical methods are used. For sparse matrices and when we only want the eigenvectors for the largest eigenvalues, the so-called **Lanczos method** is used

    It has complexity $\mathbf{O(k \cdot nnz)}$, where **nnz** is the number of nonzero values in the given matrix and **k** is the number of the largest eigenvalues to consider

    Note that for term-document matrices $nnz \ll n \cdot m$

    scipy uses a simpler (less efficient) method, see slide 18

Computing the SVD 2/7

- The eigenvalue decomposition (EVD)
  - A real symmetric $n \times n$ matrix B has n pairwise orthogonal unit eigenvectors $u_1, \ldots, u_n$ (with eigenvalues $\lambda_1, \ldots, \lambda_n$)

    That is, $B \cdot u_i = \lambda_i \cdot u_i$ and $u_i \bullet u_j = 0$, for $i \neq j$, and $|u_i| = 1$

    Equivalently, $B = U \cdot \mathrm{diag}(\lambda_1, \ldots, \lambda_n) \cdot U^T$ … and $U \cdot U^T = I$
  - For special matrices, we can often guess the eigenvectors and then compute the eigenvalues for them, for example:

$$B = \begin{pmatrix} 2 & 5 \\ 5 & 2 \end{pmatrix} \quad 2 \times 2 \qquad \boxed{u_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}} \qquad B \cdot u_1 = \begin{pmatrix} 2 & 5 \\ 5 & 2 \end{pmatrix}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 7 \\ 7 \end{pmatrix} = \underline{7} \cdot u_1$$

eigenvalue $\lambda_1 = 7$

$$U = [\bar{u}_1 \, \bar{u}_2] \qquad S = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

Normalized versions of u_1 and u_2

$$\bar{u}_1 = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \bar{u}_2 = \frac{1}{\sqrt{2}}\begin{pmatrix} -1 \\ 1 \end{pmatrix} \qquad \boxed{u_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}} \qquad B \cdot u_2 = \begin{pmatrix} 2 & 5 \\ 5 & 2 \end{pmatrix}\begin{pmatrix} -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ -3 \end{pmatrix} = \underline{-3} \cdot u_2$$

eigenvalue $\lambda_2 = -3$

$$\begin{pmatrix} 2 & 5 \\ 5 & 2 \end{pmatrix} = B = U \cdot S \cdot U^T = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}\begin{pmatrix} 7 & 0 \\ 0 & -3 \end{pmatrix} \cdot \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$$

Computing the SVD 3/7

p, q are zeros

$(\lambda - p)(\lambda - q) = \lambda^2 - (p+q)\lambda + p \cdot q$

- Computing the EVD via the determinant

  λ is the variable here

  - We can always compute all eigenvalues systematically, by solving the polynomial equation **det(B – λ · I) = 0** for **λ**
  - Let us first prove that this is correct and then apply this to the matrix from the previous slide

Let λ be an eigenvalue $\Rightarrow \exists u \neq 0 : B \cdot u = \lambda \cdot u$

$\Rightarrow B u = \lambda I \cdot u \Rightarrow (B - \lambda I) \cdot \underset{\neq 0}{u} = 0 \Rightarrow \det(B - \lambda I) = 0$

$B = \begin{pmatrix} 2 & 5 \\ 5 & 2 \end{pmatrix}, \quad B - \lambda I = \begin{pmatrix} 2-\lambda & 5 \\ 5 & 2-\lambda \end{pmatrix}$

$\det \begin{pmatrix} 2-\lambda & 5 \\ 5 & 2-\lambda \end{pmatrix} = (2-\lambda)^2 - 5^2 = \lambda^2 - 4\lambda - 21 = (*)$

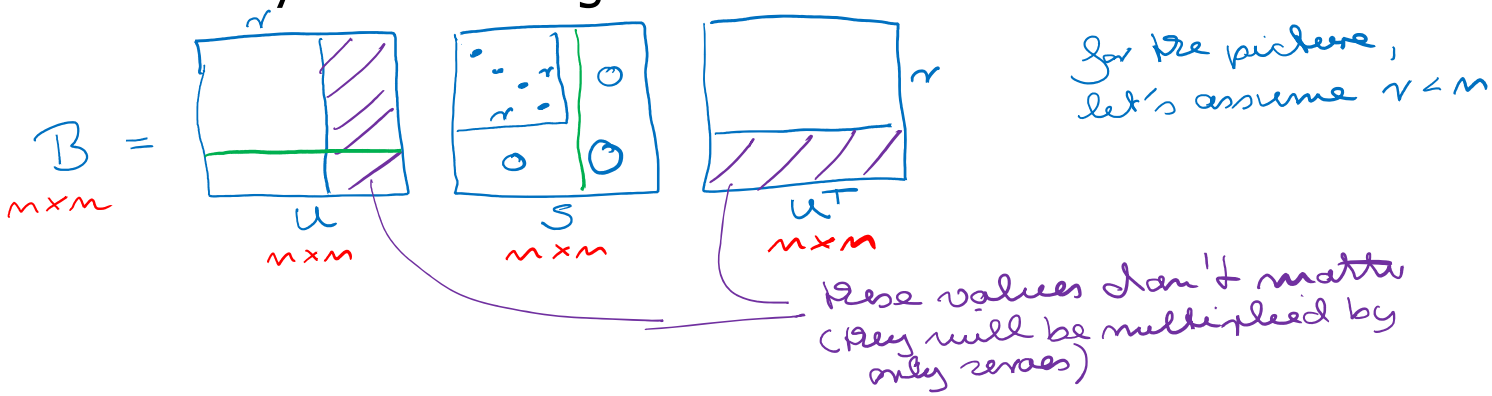
Find p, q s.t. $p \cdot q = -21$ and $p + q = 4 \Rightarrow p = 7, q = -3$

are two eigenvalues

$(*) = (\lambda - 7)(\lambda + 3)$

Computing the SVD 4/7

- Reduced form of the EVD
  - Let B be an n x n matrix with rank r ≤ n
  - Then exactly r of the diagonal entries of S are nonzero



  - We can then write the EVD in a reduced form, with an n x r matrix U (consisting of the eigenvectors of U that correspond to the nonzero eigenvalues) and an r x r matrix S

$$\underset{n\times n}{B} = \underset{n\times r}{U} \cdot \underset{r\times r}{S} \cdot \underset{r\times n}{U^T}$$

In the following, we always use this reduced form

Computing the SVD 5/7

in general:
$(X \cdot Y)^T = Y^T \cdot X^T$

- From EVD to SVD, part 1
  - Let $\mathbf{A}$ be an arbitrary m x n matrix with rank r
    $m \times n$
  - Then the matrix $\mathbf{A} \cdot \mathbf{A}^T$ is square (m x m) and symmetric
    $\underbrace{(A \cdot A^T)}_{m \times m}{}^T = A^{T^T} \cdot A^T = A \cdot A^T$ ∎
  - Hence, $\mathbf{A} \cdot \mathbf{A}^T$ has a reduced EVD with an m x r matrix U and a diagonal r x r matrix S_1

    $$\underbrace{A \cdot A^T}_{m \times m} = \underbrace{U}_{m \times r} \cdot \underbrace{S_1}_{r \times r} \cdot \underbrace{U^T}_{r \times m}$$
  - Analogously, the matrix $\mathbf{A}^T \cdot \mathbf{A}$ is square (n x n) and symmetric and has a reduced EVD with an n x r matrix V and a diagonal r x r matrix S_2

    $$\underbrace{A^T \cdot A}_{n \times n} = \underbrace{V}_{n \times r} \cdot \underbrace{S_2}_{r \times r} \cdot \underbrace{V^T}_{r \times n}$$

Computing the SVD 6/7

for a diagonal matrix
$S^T = S$

Let u be an eigenvector of $A \cdot A^T$
$A \cdot A^T \cdot u = \lambda u \Rightarrow A^T \cdot A \, (A^T \cdot u) = \lambda \cdot A^T \cdot u$

with
$U^T \cdot U = I$
S diagonal
$V \cdot V^T = I$

- From EVD to SVD, part 2
  - Let us assume that a decomposition **A = U · S · V** exists (SVD)
  - Then $\mathbf{A \cdot A^T = U \cdot S^2 \cdot U^T}$ and $\mathbf{A^T \cdot A = V^T \cdot S^2 \cdot V}$, that is:

    The **columns of U** are the eigenvectors of $A \cdot A^T$

    The **rows of V** are the eigenvectors of $A^T \cdot A$

    The **eigenvalues** of $A \cdot A^T$ and $A^T \cdot A$ are the same
    (that is, the two matrices S_1 and S_2 from the previous slide are the same)

    The **singular values** of A are just the square roots of these eigenvalues

$$A \cdot A^T = U \cdot S \cdot \underbrace{V \cdot V^T}_{=I} \cdot S \cdot U^T = U \cdot S^2 \cdot U^T$$

$$A^T \cdot A = V^T \cdot S \cdot \underbrace{U^T \cdot U}_{=I} \cdot S \cdot V = V^T \cdot S^2 \cdot V$$

Computing the SVD 7/7

- Computing the SVD in Python
  - For small matrices, we can use **numpy.linalg.svd**

    Beware: this takes forever on larger matrices, in particular, on the (not even very large) matrix you constructed for ES8

    Note that numpy gives us the **non-reduced** form of the SVD, with an **m x m** matrix **U** and an **n x n** matrix **V**

    (like for the reduced EVD, we can just take the first r columns of U and the first r rows of V, where r is the rank)
  - For sparse matrices, use **scipy.sparse.linalg.svds**

    Not particularly efficient for our purposes (doesn't use Lanczos, but Rayleigh–Ritz), see documentation for details

    Anyway, ES9 is purely mathematical, so no bother

Using LSI for better Retrieval 1/8

- **Variant 1:** work with $\mathbf{A_k}$ instead of A

	D_1	D_2	D_3	D_4	D_5	D_6
internet	1	1	0	1	0	0
web	1	0	1	1	0	0
surfing	1	1	1	2	1	1
beach	0	0	0	1	1	1

Our example A from the beginning

D'_1	D'_2	D'_3	D'_4	D'_5	D'_6
0.9	0.6	0.6	1.0	0.0	0.0
0.9	0.6	0.6	1.0	0.0	0.0
1.1	0.9	0.9	2.1	1.0	1.0
-0.1	0.1	0.1	0.9	1.0	1.0

best rank-2 approximation A_2

Using LSI for better Retrieval 2/8

- **Variant 1:** work with $\mathbf{A_k}$ instead of $\mathbf{A}$
  - Problem: A_k is a **dense** matrix, that is, most / all of it's $m \cdot n$ entries will be nonzero

    Typically, both m and n are very large, and then storing this matrix is infeasible (let alone compute with it)

    Example: $m = 1M$ and $n = 10M$ $\rightarrow$ $m \cdot n =$ **10 T**

Using LSI for better Retrieval 3/8

- **Variant 2:** work with $\mathbf{V_k}$ instead of with **A**
  - Recall: V_k gives us the representation of the documents in the k-dimensional "concept" space

	D_1	D_2	D_3	D_4	D_5	D_6
internet	1	1	0	1	0	0
web	1	0	1	1	0	0
surfing	1	1	1	2	1	1
beach	0	0	0	1	1	1

Our example A from the beginning

D'_1	D'_2	D'_3	D'_4	D'_5	D'_6
0.4	0.3	0.3	0.7	0.3	0.3
0.5	0.2	0.2	0.0	-0.6	-0.6

V_2 from the SVD of A

Using LSI for better Retrieval 4/8

- **Variant 2:** work with $\mathbf{V_k}$ instead of with **A**
  - V_k is again a dense matrix, that is, most or all of its $k \cdot n$ entries are nonzero
  - The original matrix A has $m' \cdot n$ nonzero entries, where m' is the average number of words in a document
  - So storing V_k instead of A is OK if $k \approx m'$ or smaller
  - Using V_k instead of A is also simpler (we don't need a sparse representation or an inverted index)
  - This is therefore a good variant to use in practice

    Example: $k = 300$ and $n = 10M \rightarrow k \cdot n =$ **3 G**

Using LSI for better Retrieval 5/8

- **Variant 2:** work with $\mathbf{V_k}$ instead of with $\mathbf{A}$
  - When using this variant, we also need to map the query to concept space
  - In the original space, the dot-product similarities are

    $$q^T \cdot \underset{m \times n}{A_k} = q^T \cdot (U_k \cdot S_k \cdot V_k) = \underbrace{(q^T \cdot U_k \cdot S_k)}_{q_k^T} \cdot V_k$$

  - So if we want to compute dot-produce similarities with V_k instead of with A_k, we can simple use

    $$\underset{1 \times k}{q_k^T} := \underset{1 \times m}{q^T} \cdot \underset{m \times k}{U_k} \cdot \underset{k \times k}{S_k}$$

  - Computing $q_k^T \cdot V_k$ requires time $\sim n \cdot k$

    In comparison: computing the similarities of q with the original documents requires time $\sim n \cdot$ #query words

Using LSI for better Retrieval 6/8

this is VERY expensive to compute and store

- **Variant 3:** expand the original documents
  - In Variant 2, we have transformed both the query and the documents to concept space
  - LSI can also be viewed as doing **document expansion** in the original space + no change in the query
  - Namely, let $\underset{m \times m}{T_k} = \underset{m \times k}{U_k} \cdot \underset{k \times m}{U_k^T}$ this is an m x m matrix
  - Then one can easily prove that $A_k = T_k \cdot A$

$$T_k \cdot A = U_k \cdot U_k^T \cdot U \cdot S \cdot V$$

$$U_k^T \cdot U = [I_k \; 0]$$

$$[I_k \; 0] \cdot S \cdot V = S_k \cdot V_k$$

Using LSI for better Retrieval 7/8

- Variant 3: expand the original documents
  - Here is some intuition for $\mathbf{T_k}$, assuming **0** or **1** entries

    In practice, we can get 0-1 entries by setting all entries in T above a certain threshold to 1, and all others to 0

The effect of this 1 is: if there is „internet" in D_i, add „web" to D'_i

		1 internet	2 web	3 surfing	4 beach
1	internet	1	1	0	0
2	web	1	1	0	0
3	surfing	0	0	1	0
4	beach	0	0	0	1

4×4

$\bullet$

D_i
1
0
1
0

4×1

=

D'_i	
1	1
1	2
1	3
0	4

4×1

Using LSI for better Retrieval 8/8

- Linear combination with original scores
  - Experience: LSI adds some useful information to the term-document matrix, but also a lot of **noise**
  - In practice, one would therefore use a linear combination of the original scores and the LSI scores:

Variant 1: $\text{scores} = \lambda \cdot q^T \cdot A + (1 - \lambda) \cdot q^T \cdot A_k$

Variant 2: $\text{scores} = \lambda \cdot q^T \cdot A + (1 - \lambda) \cdot q_k^T \cdot V_k$

Variant 3: $\text{scores} = \lambda \cdot q^T \cdot A + (1 - \lambda) \cdot q^T \cdot T_k \cdot A$

References

- Further reading
  - Textbook Chapter 18: Matrix decompositions & LSI

    http://nlp.stanford.edu/IR-book/pdf/18lsi.pdf
  - Deerwester, Dumais, Landauer, Furnas, Harshman

    Indexing by Latent Semantic Analysis, JASIS 41(6), 1990
- Web resources
  - http://en.wikipedia.org/wiki/Latent_semantic_indexing
  - http://en.wikipedia.org/wiki/Singular_value_decomposition
  - http://www.numpy.org/
  - http://www.scipy.org/